

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

Branches e merging

Aula 3

Código da aula: [DADOS]ANO1C1B4S27A3

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1

semana

25

Branches no Git

Você está aqui!

Branches e merging

semana

27

semana

20

Fundamentos
de Git

semana

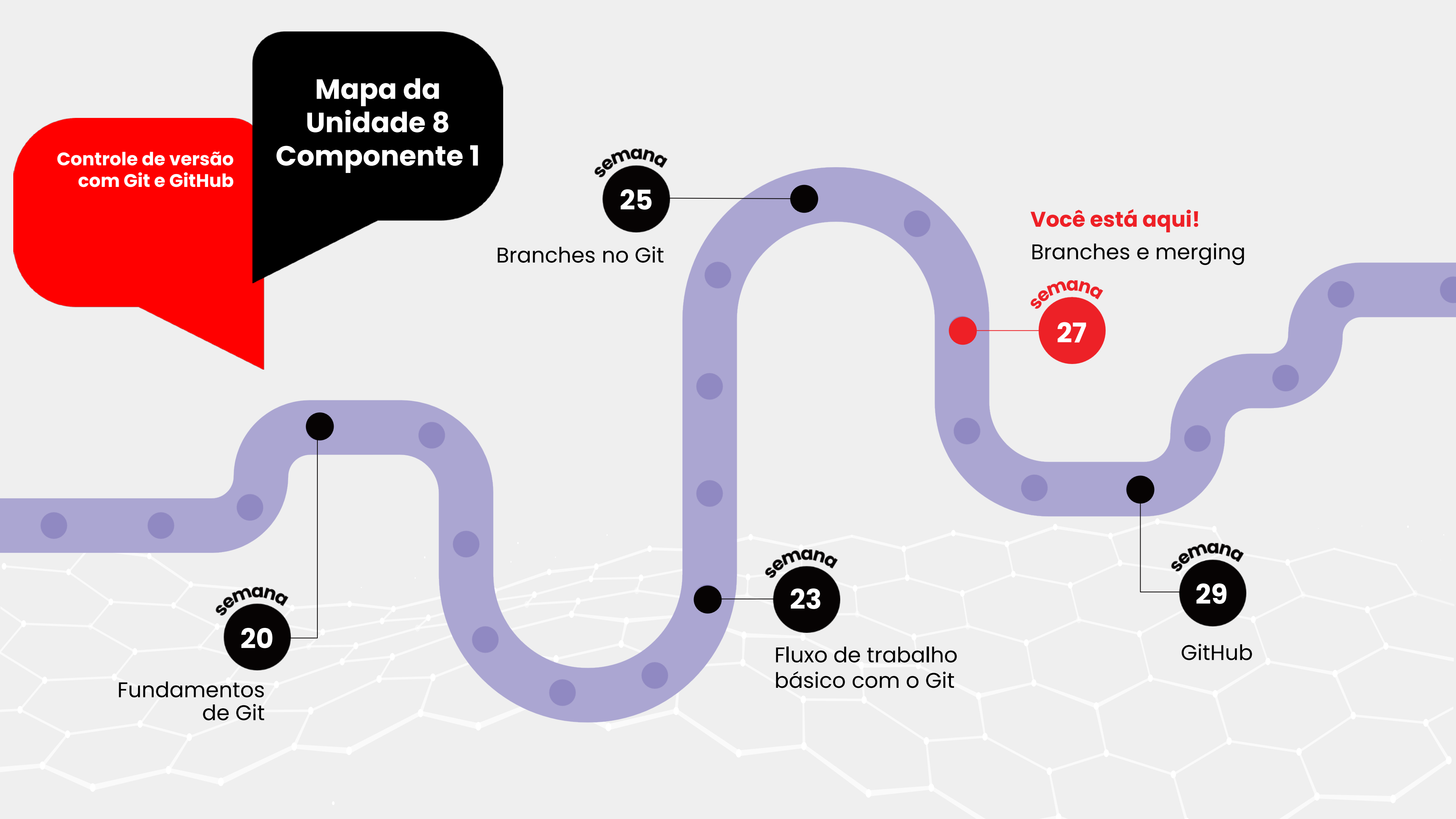
23

Fluxo de trabalho
básico com o Git

semana

29

GitHub



**Controle de versão
com Git e GitHub**

Mapa da Unidade 8 Componente 1

Você está aqui!

Branches e merging

Aula 3

Código da aula: [DADOS]ANO1C1B4S27A3

27



Objetivos da Aula

- Introduzir e utilizar ferramentas de desenvolvimento de software como Git e GitHub.



Recursos Didáticos

- Recurso audiovisual para exibição de vídeos e imagens;
- Acesso ao laboratório de informática e/ou à internet.



Duração da Aula

50 minutos.



Competências Técnicas

- Usar ferramentas de desenvolvimento de software como Git e GitHub;
- Compreender e dominar técnicas de manipulação de dados.



Competências Socioemocionais

- Analisar informações de forma crítica e tomar decisões; avaliar a qualidade dos dados e a eficácia dos modelos e das técnicas de análise;
- Colaborar efetivamente com outros profissionais, como cientistas de dados e engenheiros de dados; trabalhar em equipes multifuncionais colaborando com colegas, gestores e clientes.

Construindo
o **conceito**

Fluxos de trabalho avançado e colaboração com Git

Nesta aula, vamos explorar fluxos de trabalho colaborativos, Pull Requests, CI/CD e Git Hooks.

Esses conceitos são fundamentais para garantir a eficiência e a qualidade no desenvolvimento de software.

Construindo o **conceito**

Fluxos de trabalho colaborativo

Git Flow é um modelo de branching criado por Vincent Driessen que define um conjunto de branches específicas para organizar o desenvolvimento de software. As principais são:

- ▶ **main:** contém o histórico oficial de lançamentos.
- ▶ **develop:** contém a última versão de desenvolvimento.
- ▶ **feature:** branches usadas para desenvolver novas funcionalidades. Derivam de develop e são mescladas de volta quando a funcionalidade está completa.
- ▶ **release:** usadas para preparar uma nova versão de lançamento. Permitem ajustes finais e testes.
- ▶ **hotfix:** usadas para corrigir rapidamente bugs críticos encontrados na produção. Derivam de main e, após a correção, são mescladas de volta em main e develop.

Construindo
o **conceito**

Fluxos de trabalho colaborativo

GitHub Flow é um fluxo de trabalho simplificado, adequado para desenvolvimento contínuo e entrega contínua. Ele usa branches curtas e pequenas Pull Request (PRs) para integrar mudanças na branch main. Os passos típicos incluem:

- ▶ Criar uma branch para cada nova funcionalidade ou correção.
- ▶ Desenvolver e testar a funcionalidade.
- ▶ Abrir um PR para revisão.
- ▶ Mesclar o PR na branch main após aprovação.

Construindo
o **conceito**

Fluxos de trabalho colaborativo

GitLab Flow combina elementos do Git Flow e GitHub Flow, incorporando controle de ambiente e deploy contínuo.

Ele enfatiza a entrega contínua e a integração de branches de desenvolvimento com ambientes específicos.

Branches de produção, pré-produção e desenvolvimento são mantidas, facilitando o deploy contínuo e a integração de mudanças.

Construindo
o **conceito**

Pull Requests

- ▶ Pull Requests (PR) são uma ferramenta essencial para colaboração em projetos de software. Um PR é uma solicitação para revisar e mesclar mudanças de uma branch para outra.
- ▶ PR facilitam a revisão de código, garantindo que as mudanças sejam revisadas por outros desenvolvedores antes de serem integradas.

Construindo
o **conceito**

Pull Requests

Revisão de código é um processo em que outros desenvolvedores revisam as mudanças propostas em um PR.

Isso ajuda a identificar bugs, melhorar a qualidade do código e compartilhar conhecimento. Ferramentas como GitHub, GitLab e Bitbucket fornecem interfaces para criar e gerenciar PR, facilitando a colaboração.

Construindo
o **conceito**

Pull Requests

Comandos Git como `git fetch`, `git checkout`, e `git merge` são usados para aplicar mudanças de PR localmente.

Por exemplo, `git fetch origin pull/ID/head:BRANCHNAME` traz as mudanças de um PR específico para uma nova branch local.

Construindo
o **conceito**

Continuous Integration (CI) e Continuous Deployment (CD)

- ▶ **CI** é a prática de integrar mudanças de código frequentemente e verificar automaticamente sua integridade. Isso é feito através de pipelines automatizados que constroem, testam e validam o código sempre que há novas mudanças.
- ▶ **CD** leva CI um passo adiante, automatizando o deploy de mudanças para ambientes de produção. Isso garante que o código esteja sempre em um estado pronto para produção.

Construindo
o **conceito**

Continuous Integration (CI) e Continuous Deployment (CD)

Ferramentas populares de CI/CD incluem Jenkins, Travis CI, CircleCI e GitLab CI/CD. Essas ferramentas automatizam a construção, o teste e o deploy do código.

Um arquivo de configuração (como `.gitlab-ci.yml` ou `.travis.yml`) define os estágios do pipeline de CI/CD, incluindo construção, teste e deploy. Esses arquivos especificam scripts a serem executados em cada estágio.

Construindo
o **conceito**

Continuous Integration (CI) e Continuous Deployment (CD)

Por exemplo, um pipeline de CI/CD típico pode incluir estágios para:

Build: compilar o código.

Test: executar testes automatizados.

Deploy: implantar a aplicação em um ambiente de teste ou produção.

Construindo
o **conceito**

Exemplo: Git Flow

1. Inicializar um novo repositório e configurar Git Flow:

```
!git init myrepo  
%cd myrepo  
!git flow init -d
```

2. Criar uma branch de feature e desenvolver a funcionalidade:

```
!git flow feature start new-feature
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: Git Flow

3. Desenvolver a funcionalidade em Python:

```
# Desenvolver a funcionalidade em Python
def new_feature():
    print("This is a new feature!")

# Exemplo de uso
new_feature()
```

4. Commitar mudanças:

```
!git add .
!git commit -m "Add new feature"
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: Git Flow

5. Finalizar a feature e mesclar em develop:

```
!git flow feature finish new-feature
```

Resultado:

A nova funcionalidade é desenvolvida em uma branch de feature e mesclada na branch develop.

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: criando e revisando um Pull Request

1. Inicializar um repositório Git:

```
!git init myrepo  
%cd myrepo
```

2. Criar uma branch e desenvolver uma correção:

```
!git checkout -b bugfix/fix-issue
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: criando e revisando um Pull Request

3. Corrigir o problema em Python:

```
# Corrigir o problema em Python
def fix_issue():
    print("Issue fixed!")

# Exemplo de uso
fix_issue()
```

4. Commitar mudanças:

```
!git add .
!git commit -m "Fix issue"
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: criando e revisando um Pull Request

5. Mesclar a branch de correção na branch principal (main):

```
!git checkout main  
!git merge bugfix/fix-issue
```

Resultado:

A correção é desenvolvida em uma branch separada e mesclada na branch main.

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: configurando CI/CD com GitLab CI

1. Inicializar um repositório Git e criar a branch main:

```
!git init myrepo  
%cd myrepo  
!git checkout -b main
```

Elaborado especialmente para o curso com o prompt de comando.

Construindo
o **conceito**

Exemplo: configurando CI/CD com GitLab CI

2. Criar um arquivo de configuração para GitLab CI:

```
%%writefile .gitlab-ci.yml
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - echo "Building the application..."
    - # comandos de build aqui

test:
  stage: test
  script:
    - echo "Running tests..."
    - # comandos de teste aqui

deploy:
  stage: deploy
  script:
    - echo "Deploying the application..."
    - # comandos de deploy aqui
```


Construindo
o **conceito**

Exemplo: configurando CI/CD com GitLab CIt

3. Commitar o arquivo de configuração:

```
!git add .gitlab-ci.yml  
!git commit -m "Add GitLab CI configuration"
```

Elaborado especialmente para o curso com o prompt de comando.

Colocando
em **prática**

Exercício: interpretação de código

Instruções

1. Implemente os códigos ao lado.
2. Comente e explique o que cada linha está fazendo.
3. Registre em um documento-texto e, ao final, envie o código de programação e as respostas pelo Ambiente Virtual de Aprendizagem.



20 minutos



Duplas

```
# Inicializar um novo repositório e configurar Git Flow
!git init myrepo
%cd myrepo
!git flow init -d

# Criar uma branch de feature e desenvolver a funcionalidade
!git flow feature start new-feature

# Desenvolver a funcionalidade em Python
def new_feature():
    print("This is a new feature!")

# Exemplo de uso
new_feature()

# Commitar mudanças
!git add .
!git commit -m "Add new feature"

# Finalizar a feature e mesclar em develop
!git flow feature finish new-feature
```

Elaborado especialmente para o curso com o prompt de comando.

Colocando
em **prática**

Exercício: interpretação de código

Instruções

1. Implemente os códigos ao lado.
2. Comente e explique o que cada linha está fazendo.
3. Registre em um documento-texto e, ao final, envie o código de programação e as respostas pelo Ambiente Virtual de Aprendizagem.



20 minutos



Duplas

```
# Inicializar um repositório Git
!git init myrepo_pr
%cd myrepo_pr

# Criar um commit inicial
!echo "Initial commit" > README.md
!git add README.md
!git commit -m "Initial commit"

# Criar uma branch e desenvolver uma correção
!git checkout -b bugfix/fix-issue

# Corrigir o problema em Python
def fix_issue():
    print("Issue fixed!")

# Exemplo de uso
fix_issue()

# Commitar mudanças
!git add .
!git commit -m "Fix issue"

# Criar a branch principal main
!git checkout -b main

# Mesclar a branch de correção na branch principal
!git merge bugfix/fix-issue
```

Elaborado especialmente para o curso com o prompt de comando.

Ser
sempre +

Situação

Você é um desenvolvedor em uma equipe de software que está trabalhando em um grande projeto.

De repente, você percebe que uma funcionalidade crítica desenvolvida por outro colega tem um bug que pode causar falhas significativas no sistema. Seu colega está de férias e não estará disponível para corrigir o problema antes da próxima release.

O prazo para a entrega está se aproximando rapidamente.

Situação fictícia elaborada especialmente para o curso.

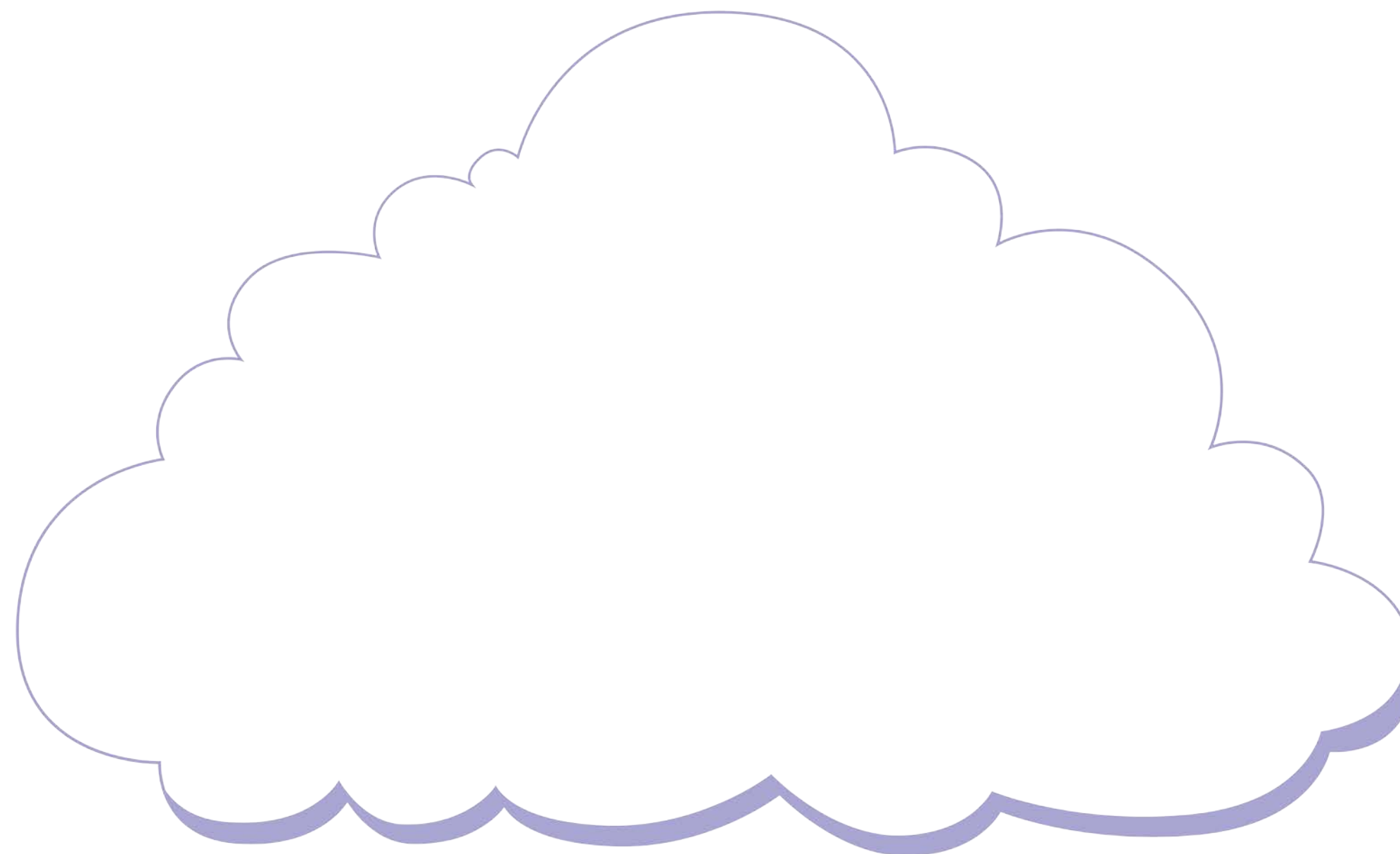
Ser
sempre +

Ação

Perguntas para reflexão e ação:

1. Quais são os primeiros passos que você deve tomar ao descobrir o bug?
2. Como você deve comunicar a situação à sua equipe e aos gestores?
3. Quais estratégias você pode usar para garantir que o problema seja resolvido eficientemente e que a entrega não seja comprometida?

Nuvem de palavras



© Getty Images

O que nós
**aprendemos
hoje?**



© Getty Images

O que nós
**aprendemos
hoje?**

Então ficamos assim...

- 1** Exploramos fluxos de trabalho colaborativos no Git, incluindo Git Flow, GitHub Flow e GitLab Flow. Cada um tem suas próprias práticas e convenções para organizar branches e facilitar a colaboração.
- 2** Discutimos Pull Requests como ferramentas essenciais para revisão de código e integração. Revisão de código melhora a qualidade do software e facilita a colaboração em equipe.
- 3** Introduzimos CI/CD e Git Hooks para automação e controle de qualidade. CI/CD automatiza construção, teste e deploy, enquanto Git Hooks automatizam tarefas específicas em eventos Git.

Saiba mais

Controle de versão nunca foi tão fácil!

Junte o conhecimento da aula e o curso disponível no link abaixo e seja imbatível na organização de projetos!

ALURA. *Git e GitHub: dominando o controle de versão de código*. Disponível em: <https://www.alura.com.br/curso-online-git-github-dominando-controle-versao-codigo>.

Acesso em: 24 jul. 2024.

Referências da aula

BELL, P.; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2015.

GITHUB DOCS. *Documentação de introdução ao GitHub*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started>. Acesso em: 23 jul. 2024.

Identidade visual: imagens © Getty Images

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**